MINGGU KE-9 • PEMROGRAMAN DAN PENGUJIAN APLIKASI WEB (12S3101)

Web Services: Konsep Layanan, Arsitektur SOAP, & RESTful Services

Integrasi Sistem Terdistribusi, Kontrak API, Protokol SOAP/WSDL, Semantik REST, hingga Pengujian API Nyata

Dosen Pengampu: Chandro Pardede, S.Kom., M.Sc. • **Pemrograman dan Pengujian Aplikasi Web (12S3101)**
Institut Teknologi Del • FITE • Program Studi S1 Sistem Informasi

CAPAIAN PEMBELAJARAN (OBJECTIVES) MINGGU KE-9

POIN PEMBAHASAN UTAMA:

- Menjelaskan motivasi penggunaan Web Service pada sistem terdistribusi
- Menjabarkan konsep Service, Web Service, API, dan Service Contract
- Membedakan perspektif Service Consumer dan Service Provider
- Memahami cara kerja SOAP-Based Services (SOAP XML, WSDL, & UDDI)
- Menguasai konsep REST-Based Services (Semantik HTTP, Stateless, Token)

SUB-CPMK MINGGU 9 (RPS 12S3101):

Mahasiswa mampu menganalisis kebutuhan dan kelayakan (feasibility) penerapan Web Service serta menggunakan layanan berbasis REST dengan tepat.

Penyatuan Utuh 3 Modul Minggu 9:

```
[Modul 09-01: Web Services Fundamental]
  ├── Motivasi, Orkestrasi, & Kontrak API
  └── Perspektif Consumer vs Provider
[Modul 09-02: SOAP-Based Services]
  ├── Spesifikasi W3C: SOAP, WSDL, UDDI
  └── Struktur Pesan XML & Alur Kerja SOAP
[Modul 09-03: REST-Based Services]
  ├── Segitiga REST, Semantik, & Stateless
  └── Studi Kasus GitHub API & Token Auth
```

KORELASI MATERI MINGGU 1–6 MENUJU MINGGU 9

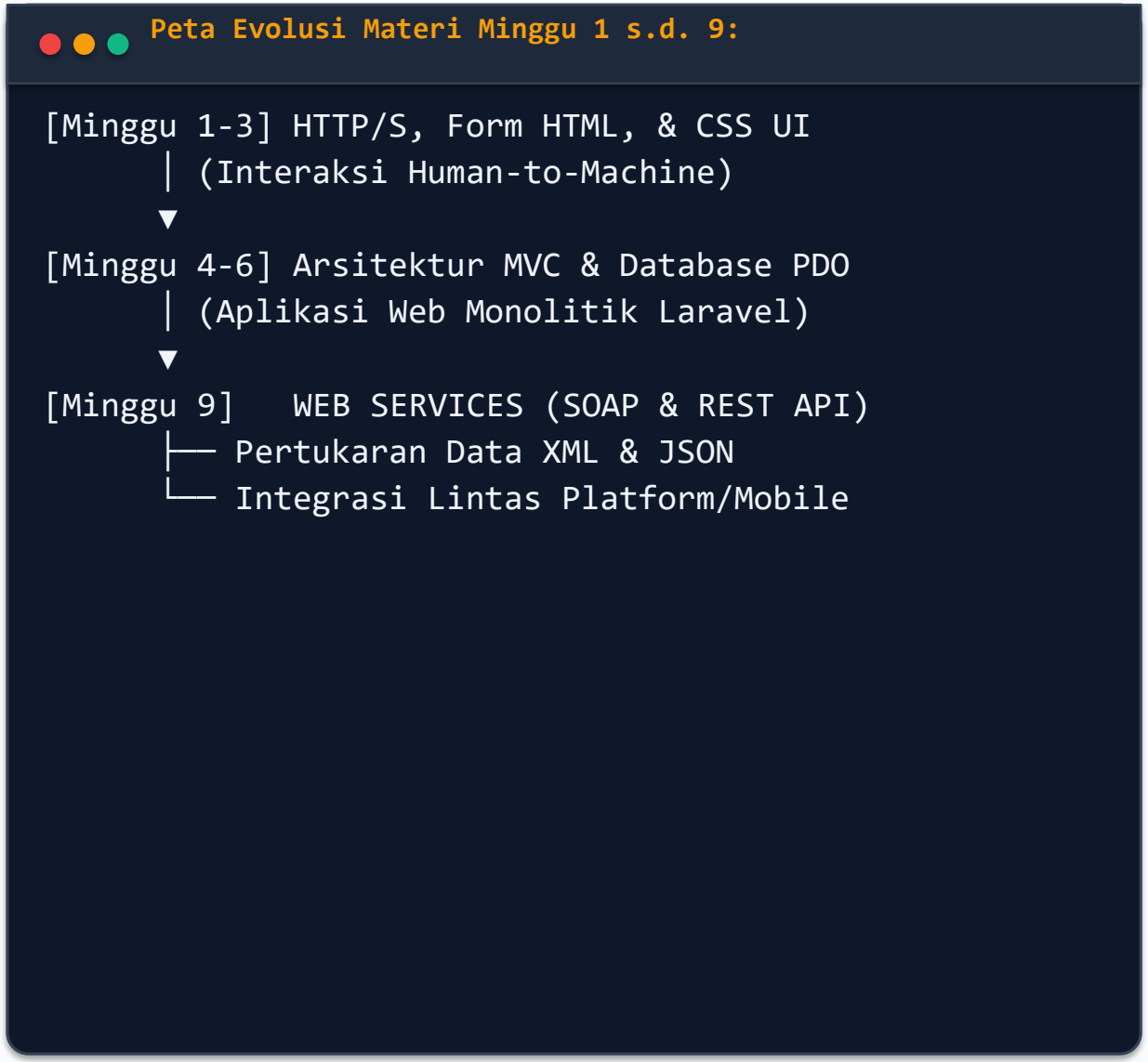
POIN PEMBAHASAN UTAMA:

- Minggu 1 (HTTP/S): Paket Request & Response menjadi jalur Web Service
- Minggu 2–3 (HTML & CSS): Dari tampilan manusia ke pertukaran data mesin
- Minggu 4 (Arsitektur & State): Dari Session/Cookie ke Stateless Token
- Minggu 5 (MVC Laravel): Controller kini dapat mereturn JSON selain View
- Minggu 6 (Database & ORM): Eloquent & ACID menjadi logika di balik API

TRANSFORMASI ARSITEKTUR PASCA-UTS:

Sebelum UTS kita membangun aplikasi web monolitik untuk diakses browser manusia. Mulai Minggu 9, aplikasi kita belajar berbicara dengan aplikasi lain!

Peta Evolusi Materi Minggu 1 s.d. 9:



PETA ALUR PEMBAHASAN (ROADMAP)

POIN PEMBAHASAN UTAMA:

- Bagian 1–2: Objective, Korelasi Materi, & Motivasi Sistem Terdistribusi
- Bagian 3: Definisi Service, Web Service, Kontrak API, & Kapan Dipakai
- Bagian 4: Arsitektur SOAP-Based Services (SOAP XML, WSDL, Workflow)
- Bagian 5: Arsitektur REST-Based Services (Segitiga REST, Stateless)
- Bagian 6–7: Studi Kasus GitHub REST API, Integrasi Laravel, & Evaluasi

ARAH PROYEK MATA KULIAH:

Kita mempelajari SOAP untuk memahami standar sistem enterprise/legacy, namun proyek pengembangan dan pengujian API Anda akan berfokus pada REST.

Sistematika 38 Halaman Slide:

```
Bagian 1 (01-04) : Objective & Jembatan Materi
Bagian 2 (05-08) : Motivasi & Orkestrasi Layanan
Bagian 3 (09-14) : Konsep Service & Kontrak API
Bagian 4 (15-21) : Arsitektur SOAP & WSDL
Bagian 5 (22-28) : Arsitektur REST & Stateless
Bagian 6 (29-35) : Studi Kasus GitHub & Laravel
Bagian 7 (36-38) : To-Dos, Tugas, & Referensi
```

MOTIVASI: PRINSIP NO SUPER-ENTERPRISE & DOMAIN FOCUS

POIN PEMBAHASAN UTAMA:

- No super-enterprise, no super-app yang mengerjakan semuanya sendiri
- Setiap aplikasi wajib fokus pada bisnis intinya (core business domain)
- Fungsi di luar kepakaran domain sebaiknya tidak dibangun dari nol
- Membangun semua fitur sendiri memicu biaya tinggi dan rawan celah bug
- Solusinya adalah kolaborasi antar-aplikasi melalui pertukaran layanan

FOKUS PADA KEPAKARAN DOMAIN:

Aplikasi kampus (SIAKAD) harus fokus mengelola KRS dan nilai akademik, bukan membangun mesin pencetak nomor Virtual Account bank sendiri.

Monolitik Terisolasi vs Fokus Domain:

```
[PENDEKATAN KELIRU: Buat Semua Sendiri]
Aplikasi Toko Online mencoba membuat:
  ✗ Server Email & SMS Gateway sendiri
  ✗ Mesin Otorisasi Kartu Kredit sendiri
  ✗ Sistem Pelacakan Kurir GPS sendiri

[PENDEKATAN TEPAT: Fokus Bisnis Inti]
Aplikasi Toko Online fokus pada Katalog & Order,
lalu terhubung ke layanan eksternal via API!
```

KEBUTUHAN PERTUKARAN DATA & KAPABILITAS TERDISTRIBUSI

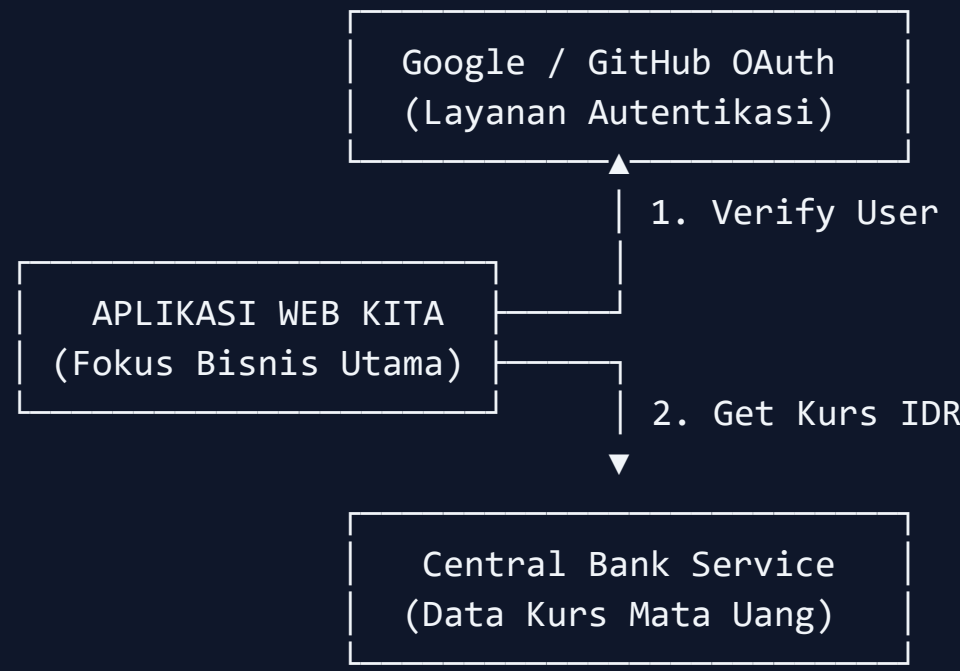
POIN PEMBAHASAN UTAMA:

- Aplikasi modern membutuhkan pertukaran data lintas organisasi
- Kapabilitas milik pihak lain harus dapat diakses secara terprogram
- Contoh 1 (Autentikasi): Login via layanan OAuth Google atau GitHub
- Contoh 2 (Keuangan): Kurs nilai tukar uang dari layanan Bank Sentral
- Contoh 3 (Infrastruktur): Peta lokasi dari Google Maps API

KORELASI DENGAN DATABASE MINGGU 6:

Di Minggu 6 aplikasi membaca database lokalnya sendiri. Namun kita tidak mungkin mengakses port 3306 database milik Google atau Bank Sentral secara langsung!

Peta Pertukaran Kapabilitas Lintas Sistem:



ILUSTRASI ORKESTRASI LAYANAN (SERVICE ORCHESTRATION)

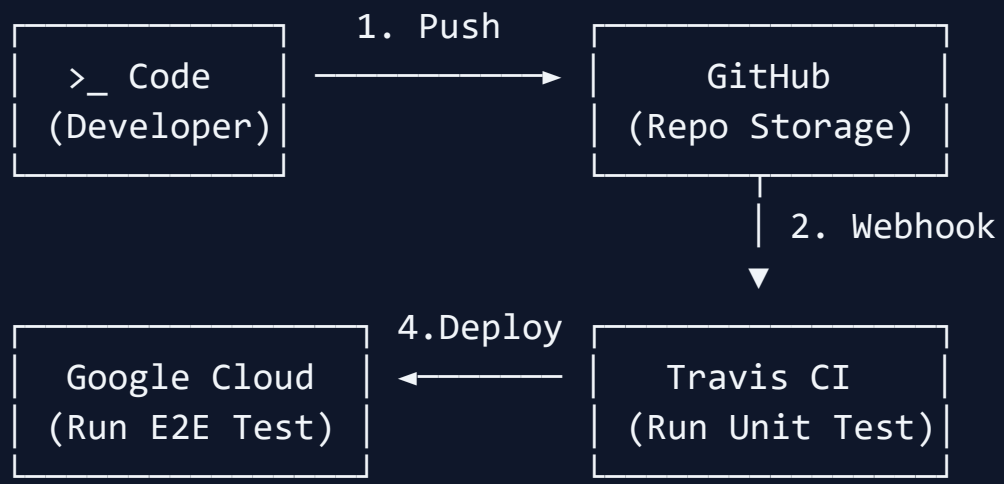
POIN PEMBAHASAN UTAMA:

- Analogi: Setiap individu memiliki tanggung jawab spesifik dalam hidup
- Tanggung jawab tersebut dirangkai (chained / orchestrated) berurutan
- Rangkaian ini menciptakan layanan baru yang lebih bernilai tinggi
- Setiap pihak dalam orkestrasi wajib menjalankan tugasnya dengan benar
- Ilustrasi Pipeline: Code -> GitHub -> Travis CI -> Google Cloud

KEANDALAN RANTAI ORKESTRASI:

Jika satu komponen dalam rantai orkestrasi gagal memenuhi kontrak kerjanya, maka tujuan akhir dari keseluruhan layanan kompleks tidak akan tercapai.

Ilustrasi Pipeline Otomatis (Slide 09-01):



ORKESTRASI EKOSISTEM DIGITAL: GOFOOD & TOKOPEDIA

POIN PEMBAHASAN UTAMA:

- GoFood & Tokopedia adalah contoh nyata orkestrasi layanan di Indonesia
- Saat pengguna memesan makanan, aplikasi mengorkestrasi banyak service
- Layanan Restoran memeriksa stok menu dan menghitung total harga
- Layanan Pembayaran (GoPay/Bank) memotong saldo secara atomik
- Layanan Maps & Driver mencarikan kurir terdekat dan melacak rute

NILAI TAMBAH ORKESTRASI:

Pengguna merasakan satu pengalaman utuh yang mulus, padahal di balik layar terjadi koordinasi puluhan layanan independen dalam hitungan milidetik.

Orkestrasi Transaksi pada GoFood / Tokopedia:

```
[User Klik 'Pesan Sekarang' di Aplikasi]
    |
    ├─► 1. Merchant Service  : Cek stok & harga
    ├─► 2. Promo Service     : Hitung diskon
    ├─► 3. Payment Service   : Potong saldo GoPay
    ├─► 4. Dispatch Service  : Cari mitra kurir
    └─► 5. Notification Svc  : Kirim push notif
    |
    ▼
[Pesanan Tiba: Nilai Layanan Tinggi Tercipta!]
```

APA ITU SERVICE (LAYANAN PERANGKAT LUNAK)?

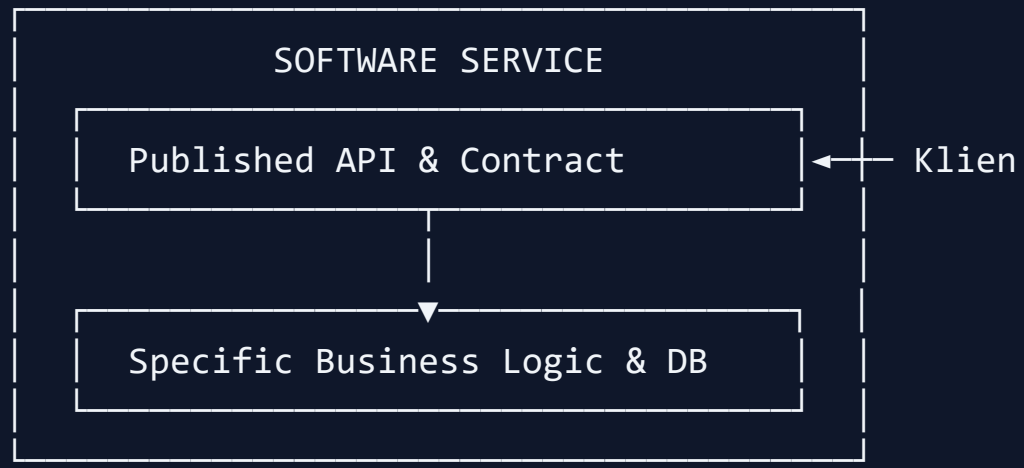
POIN PEMBAHASAN UTAMA:

- Service adalah suatu fungsionalitas dengan tujuan bisnis yang spesifik
- Dalam konteks rekayasa perangkat lunak, service berwujud program/kode
- Fungsionalitasnya dibuka agar dapat diakses lewat API yang terdefinisi
- API (Application Programming Interface) menjadi pintu komunikasi resmi
- Seluruh aturan pemanggilan API tersebut dituangkan di dalam Kontrak

PERBEDAAN LIBRARY VS SERVICE:

Library (Minggu 7) di-import dan berjalan di dalam memori aplikasi kita sendiri, sedangkan Service berjalan di proses/server terpisah dan dipanggil lewat jaringan.

Anatomi Dasar Sebuah Software Service:



APA ITU WEB SERVICE? (KORELASI HTTP MINGGU 1)

POIN PEMBAHASAN UTAMA:

- Web Service adalah service yang disediakan melalui protokol web
- Menggunakan protokol HTTP / HTTPS beserta standar-standarnya
- Memungkinkan aplikasi beda bahasa (PHP, Go, Java) saling bicara
- Tidak mengembalikan halaman HTML visual, melainkan data murni
- Mayoritas diimplementasikan dalam bentuk layanan SOAP atau REST

MENGAPA MENGGUNAKAN PROTOKOL HTTP/S?

Seperti dipelajari di Minggu 1, port HTTP (80) dan HTTPS (443) sudah didukung oleh seluruh infrastruktur internet dan firewall di seluruh dunia.

Web App Tradisional vs Web Service via HTTP:

```
// 1. Web App Tradisional (Minggu 2 & 5):
Browser ──(HTTP GET /krs)──► Server Laravel
Browser ◄──(HTML + CSS)─── Server Laravel

// 2. Web Service (Minggu 9):
Client ──(HTTP GET /api/krs)──► Web Service
Client ◄──(XML / JSON Data)─── Web Service

Dua Arsitektur Utama Web Service:
  1. SOAP-Based Services (Protokol XML W3C)
  2. REST-Based Services (Semantik HTTP)
```

APA ITU KONTRAK LAYANAN (SERVICE CONTRACT)?

POIN PEMBAHASAN UTAMA:

- Sebuah service diakses oleh pengguna layanan (consumer) via API
- Spesifikasi API didefinisikan secara eksplisit di dalam Kontrak
- Isi 1: Bagaimana cara mengonsumsi API (URL, metode, & header)
- Isi 2: Spesifikasi parameter input (nama, tipe, wajib/opsional)
- Isi 3: Ekspektasi output yang dihasilkan (format & kode status)

KONTRAK SEBAGAI KESEPAKATAN MENGIKAT:

Baik SOAP maupun REST memiliki cara berbeda untuk mempublikasikan kontraknya, namun keduanya wajib mematuhi kontrak agar klien tidak mengalami error.

Tiga Komponen Utama dalam Service Contract:

```
┌──────────────────────────────────────────────┐
│          API SERVICE CONTRACT                │
├──────────────────────────────────────────────┤
│ 1. HOW TO CONSUME:                           │
│    Method : GET                              │
│    URL    : https://api.del.ac.id/v1/mhs     │
├──────────────────────────────────────────────┤
│ 2. PARAMETER SPECIFICATIONS:                 │
│    Query  : nim (string, 8 chars, wajib)     │
├──────────────────────────────────────────────┤
│ 3. EXPECTED OUTPUT:                          │
│    Status : 200 OK (application/json)        │
│    Body   : {"nim":"12S21001","ipk":3.85}    │
└──────────────────────────────────────────────┘
```

TRANSPARENT BUSINESS LOGIC (ABSTRAKSI KOTAK HITAM)

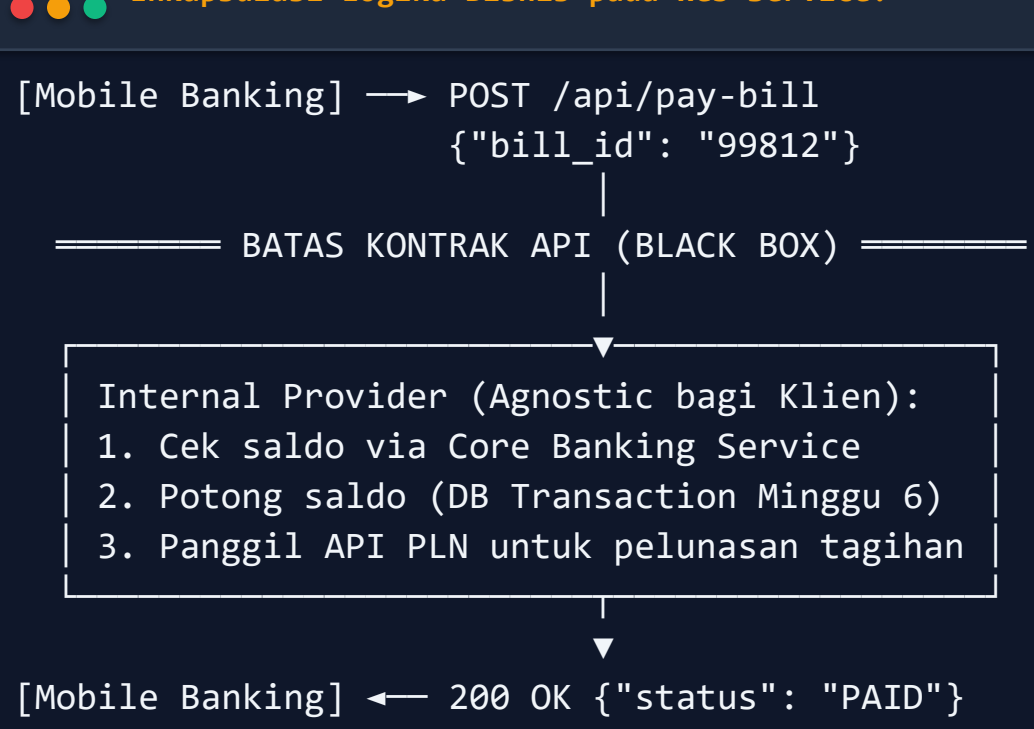
POIN PEMBAHASAN UTAMA:

- Sebuah service membungkus logika bisnis spesifik di dalamnya
- Dapat tersusun dari komposisi beberapa service yang lebih kecil
- Logika bisnis internal bersifat transparan/agnostic bagi klien
- Konsumen tidak perlu tahu bahasa kode atau tabel database-nya
- Contoh: Bayar tagihan di Mobile Banking — bagaimana? Who cares!

KORELASI DENGAN MVC & DATABASE (MINGGU 5–6):

Apakah penyedia layanan memakai Laravel Eloquent (Minggu 6) atau Golang di belakang layar, konsumen API sama sekali tidak terdampak selama kontraknya tetap sama.

Enkapsulasi Logika Bisnis pada Web Service:



PERSPEKTIF CONSUMER: KAPAN MENGONSUMSI SERVICE?

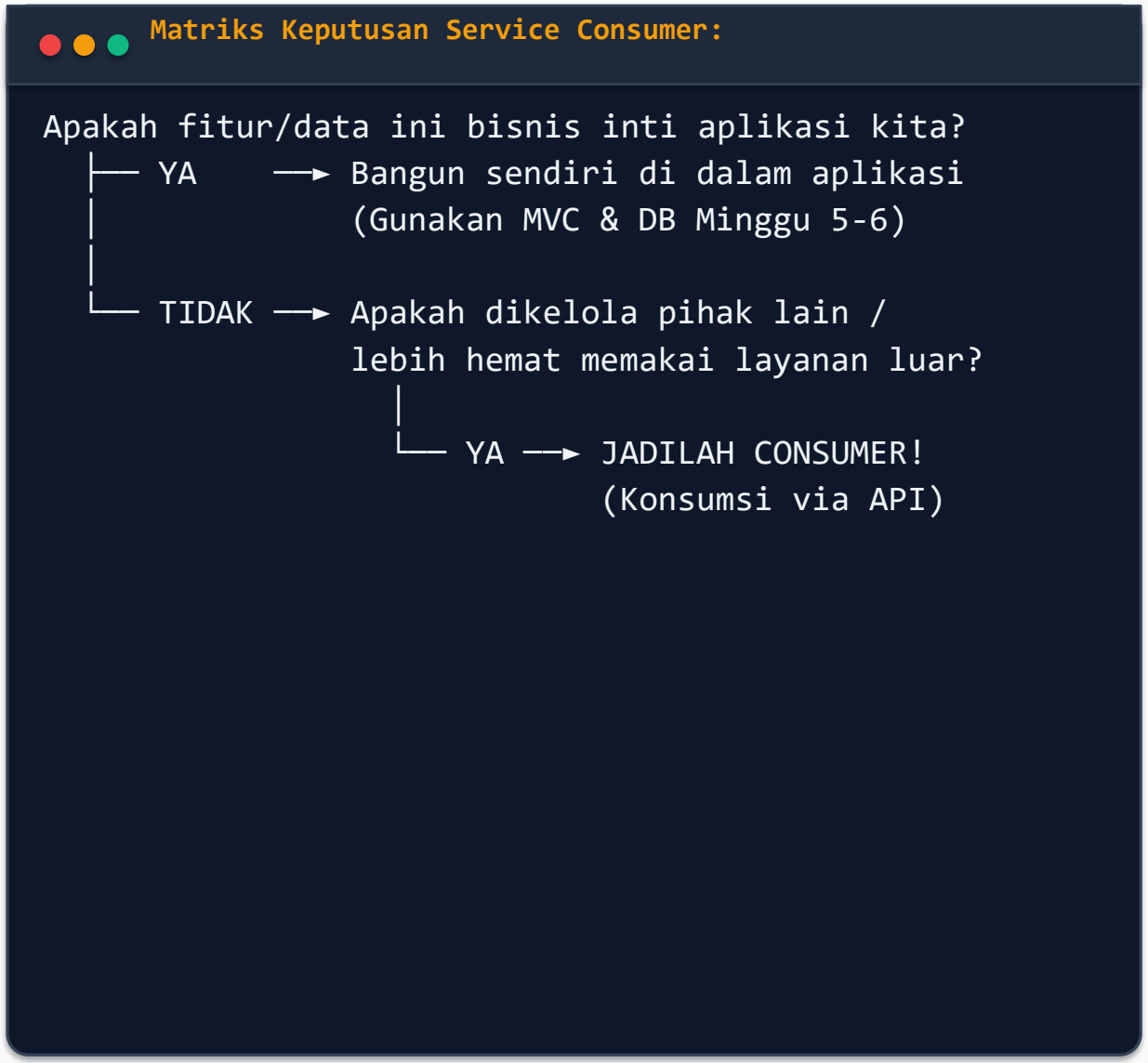
POIN PEMBAHASAN UTAMA:

- Consumer adalah pihak yang memanggil/menggunakan suatu service
- Alasan 1: Kapabilitas/data yang dibutuhkan bukan bisnis utama kita
- Alasan 2: Data tersebut memang dikelola resmi oleh aplikasi lain
- Alasan 3: Jauh lebih hemat biaya (cost-effective) dibanding buat
- Mempercepat waktu rilis produk (time-to-market) lewat integrasi

ANALISIS BUILD VS CONSUME:

Dalam rekayasa sistem informasi, keputusan menggunakan Web Service pihak ketiga selalu didasarkan pada efisiensi biaya, keandalan, dan fokus domain.

Matriks Keputusan Service Consumer:



PERSPEKTIF PROVIDER: KAPAN MENYEDIAKAN SERVICE?

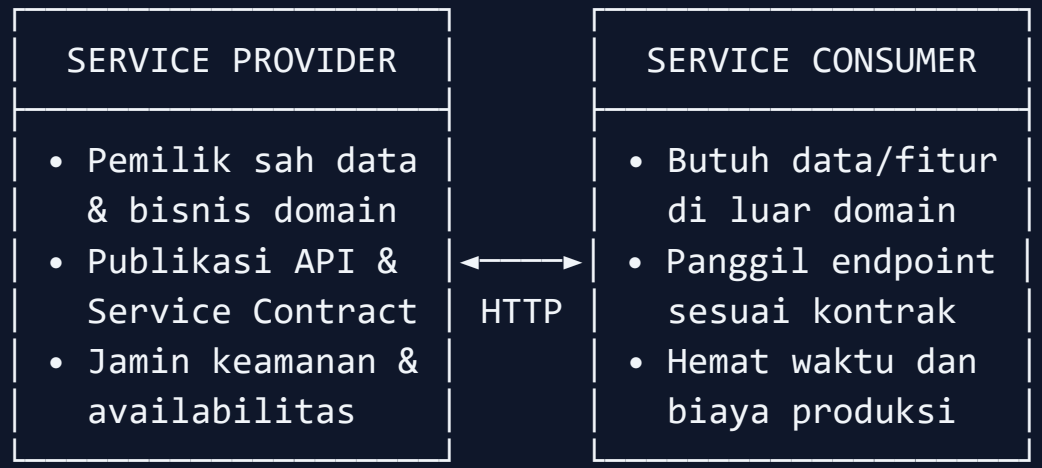
POIN PEMBAHASAN UTAMA:

- Provider adalah pihak yang menyediakan service bagi pihak lain
- Alasan 1: Data atau kapabilitas berada di dalam domain kita
- Alasan 2: Kita ingin membagikan (share) kapabilitas tersebut
- Alasan 3: Agar sistem dapat diakses oleh aplikasi Mobile / Mitra
- Satu aplikasi dapat berperan sebagai Provider sekaligus Consumer

CONTOH PERAN GANDA PADA SIAKAD IT DEL:

SIAKAD menjadi Provider data status mahasiswa aktif bagi sistem Perpustakaan, sekaligus menjadi Consumer layanan Virtual Account milik Bank!

Hubungan Service Provider & Service Consumer:



SPESIFIKASI SOAP (SIMPLE OBJECT ACCESS PROTOCOL)

POIN PEMBAHASAN UTAMA:

- SOAP adalah singkatan dari Simple Object Access Protocol
- Merupakan layanan web yang mematuhi pedoman spesifikasi SOAP
- Spesifikasi SOAP distandarisasi oleh W3C (w3.org/TR/soap12/)
- Spesifikasi Dasar (Basic): SOAP, WSDL, dan UDDI (fokus utama)
- Spesifikasi Lanjut (Extended): WS-Security, WS-Policy, & WS-I

STANDAR PROTOKOL KETAT (W3C):

Berbeda dengan REST yang merupakan gaya arsitektur, SOAP adalah protokol resmi dengan aturan struktur XML dan keamanan tingkat enterprise (WS-*).

Ekosistem Spesifikasi W3C SOAP-Based Services:

```
┌──────────────────────────────────────────────────┐
│        SPESIFIKASI SOAP-BASED SERVICES           │
├──────────────────────────────────────────────────┤
│ 1. BASIC SPECIFICATIONS (Fokus Utama Kita):      │
│    • SOAP : Format amplop pesan berbasis XML     │
│    • WSDL : Bahasa deskripsi kontrak layanan     │
│    • UDDI : Direktori registrasi layanan         │
├──────────────────────────────────────────────────┤
│ 2. EXTENDED SPECIFICATIONS (Enterprise WS-*):    │
│    • WS-Security, WS-Policy, WS-I                │
└──────────────────────────────────────────────────┘
```

TIGA PILAR DASAR SOA: SOAP, WSDL, DAN UDDI

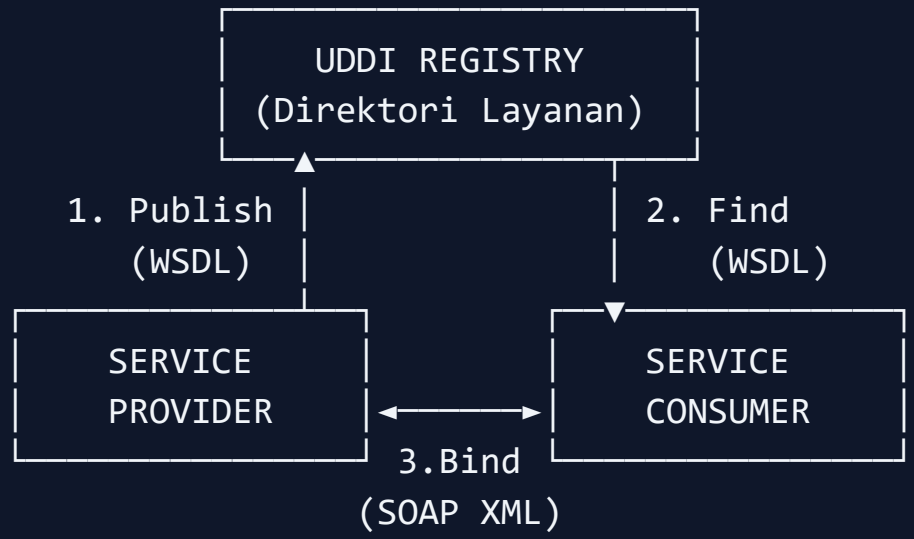
POIN PEMBAHASAN UTAMA:

- WSDL (Web Services Description Language): Kontrak XML penyedia
- UDDI (Universal Description, Discovery, Integration): Direktori
- SOAP (Simple Object Access Protocol): Protokol pesan XML
- Provider mendaftarkan kontrak WSDL ke registri UDDI (Publish)
- Consumer mencari di UDDI (Find) lalu memanggil via SOAP (Bind)

ANALOGI BUKU TELEPON (YELLOW PAGES):

UDDI ibarat buku Yellow Pages tempat perusahaan mendaftarkan layanannya, WSDL adalah brosur teknis cara pakainya, dan SOAP adalah surat resminya.

Segitiga Interaksi UDDI, WSDL, dan SOAP:



STRUKTUR PESAN SOAP (SOAP MESSAGE ENVELOPE)

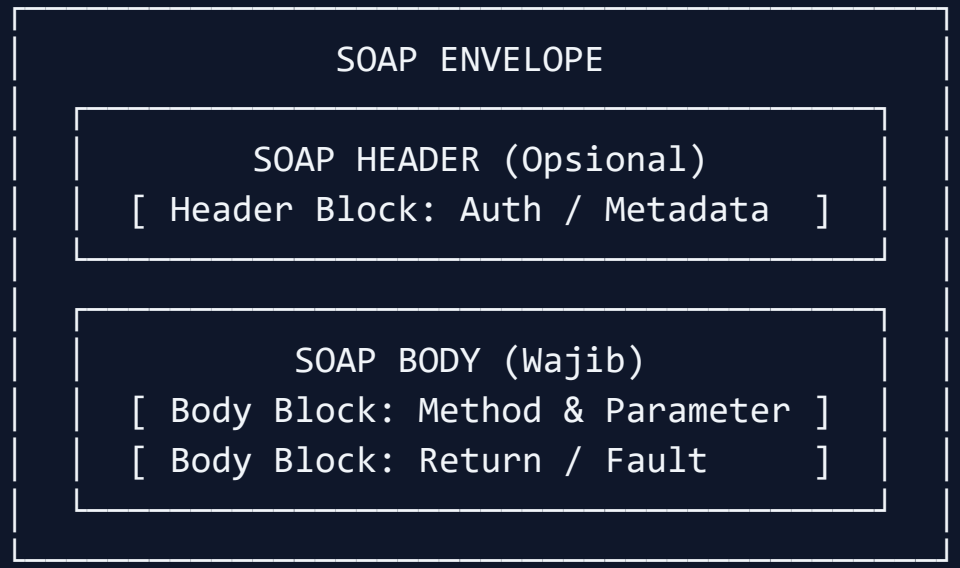
POIN PEMBAHASAN UTAMA:

- Request maupun Response pada SOAP dibungkus dalam format XML
- Struktur dokumen XML tersebut diatur ketat (well-defined XML)
- 1. SOAP Envelope: Elemen akar (root) pembungkus seluruh pesan
- 2. SOAP Header: Blok opsional berisi metadata atau keamanan
- 3. SOAP Body: Blok wajib berisi operasi, parameter, atau hasil

PENANGANAN ERROR PADA SOAP (SOAP FAULT):

Jika terjadi kegagalan di sisi server, SOAP menyediakan elemen khusus <soap:Fault> di dalam <soap:Body> untuk melaporkan kode dan deskripsi error.

Diagram Blok SOAP Envelope (Slide 09-02 Hal 8):



BEDAH CONTOH XML PESAN SOAP (GETSTOCKPRICE)

POIN PEMBAHASAN UTAMA:

- Diawali deklarasi versi XML: <?xml version="1.0"?>
- Tag <soap:Envelope> memuat namespace resmi W3C soap-envelope
- Atribut soap:encodingStyle mengatur aturan serialisasi tipe data
- Tag <soap:Body> memuat namespace domain (www.test.org/stock)
- Elemen <m:GetStockPrice> membawa StockName (IBM) & Price (126)

CIRI KHAS XML NAMESPACE (XMLNS):

Penggunaan prefix 'soap:' dan 'm:' mencegah bentrok nama tag XML antara elemen protokol standar W3C dengan elemen data bisnis aplikasi.

Contoh XML Pesan SOAP (Slide 09-02 Hal 8):

```
<?xml version="1.0"?>
<soap:Envelope
  xmlns:soap="http://www.w3.org/2003/05/
              soap-envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/
                      soap/encoding/">
  <soap:Body xmlns:m="http://www.test.org/stock">
    <m:GetStockPrice>
      <m:StockName>IBM</m:StockName>
      <m:StockPrice>126</m:StockPrice>
    </m:GetStockPrice>
  </soap:Body>
</soap:Envelope>
```

WSDL: WEB SERVICES DESCRIPTION LANGUAGE

POIN PEMBAHASAN UTAMA:

- WSDL adalah dokumen berbasis XML pendefinisi endpoint service
- Berfungsi sebagai Kontrak Formal spesifikasi lengkap layanan
- Memuat daftar operasi (method) beserta parameter inputnya
- Mendefinisikan tipe nilai balik (return value) & alamat URL
- Bersifat WAJIB (mandatory) untuk men-generate Client Stub

GENERASI STUB KLIEN OTOMATIS:

Dengan membaca file WSDL, IDE atau tool (seperti wsimport / PHP SoapClient) dapat langsung membuat kelas objek lokal (stub) tanpa koding XML manual.

Struktur Kontrak WSDL & Pemakaian di PHP:

```
<!-- Komponen Utama di dalam File .wsdl -->
<definitions name="StockService">
  <types>    <!-- Skema Data XSD   --> </types>
  <message>  <!-- Param Input/Out  --> </message>
  <portType> <!-- Daftar Operasi   --> </portType>
  <binding>  <!-- Protokol SOAP    --> </binding>
  <service>  <!-- Endpoint Address --> </service>
</definitions>

// Pemakaian WSDL di PHP (Otomatis buat Stub):
$ws = new SoapClient("http://.../stock.wsdl");
$res = $ws->GetStockPrice(["StockName"=>"IBM"]);
```

ALUR KERJA UMUM (GENERAL WORKFLOW) ARSITEKTUR SOAP

POIN PEMBAHASAN UTAMA:

- Langkah 1–2: Client App panggil Stub -> jadi SOAP Request XML
- Langkah 3: SOAP Request dikirim via HTTP menuju SOAP Engine
- Langkah 4: SOAP Engine mem-parsing XML & memanggil framework WS
- Langkah 5–6: Hasil dibungkus SOAP Engine menjadi SOAP Response
- Langkah 7–8: SOAP Client menerima XML & mereturn objek hasil

PERAN INFRASTRUKTUR STUB & SOAP ENGINE:

Aplikasi klien dan bisnis server tidak perlu mem-parsing string XML secara manual karena telah ditangani oleh lapisan SOAP Client Infrastructure dan SOAP Engine.

Diagram 8 Langkah Workflow SOAP (Slide 09-02):



POSISI SOAP SAAT INI: SISTEM LEGACY VS MODERN

POIN PEMBAHASAN UTAMA:

- Layanan berbasis SOAP umumnya ditemukan pada aplikasi legacy
- Masih dipakai di sistem inti perbankan, asuransi, & telekomunikasi
- Jarang dipilih untuk pengembangan aplikasi web/mobile baru
- Kelemahan SOAP: ukuran paket XML besar (verbose) & parsing berat
- Proyek mata kuliah kita bergerak menuju layanan berbasis REST

MENGAPA TETAP PERLU MEMAHAMI SOAP?

Sebagai lulusan Sistem Informasi, saat bekerja di korporasi/bank Anda akan sering ditugaskan mengintegrasikan aplikasi REST baru dengan sistem inti SOAP lama.

Perbandingan Ukuran Payload SOAP vs JSON:

```
// 1. Payload SOAP XML (~310 bytes):
<soap:Envelope xmlns:soap="http://www.w3.org/...">
  <soap:Body xmlns:m="http://www.test.org/stock">
    <m:GetStockPrice>
      <m:StockName>IBM</m:StockName>
      <m:StockPrice>126</m:StockPrice>
    </m:GetStockPrice>
  </soap:Body>
</soap:Envelope>

// 2. Payload REST JSON (~38 bytes - 8x Ringkas!):
{"stockName": "IBM", "stockPrice": 126}
```

APA ITU REST (REPRESENTATIONAL STATE TRANSFER)?

POIN PEMBAHASAN UTAMA:

- REST adalah singkatan dari REpresentational State Transfer
- Diperkenalkan oleh Roy Fielding dalam disertasi doktor (2000)
- REST adalah pola arsitektur (architectural pattern), bukan protokol
- Berjalan langsung di atas infrastruktur protokol HTTP/S standar
- State data dipertukarkan dalam format standar (JSON, XML, dll.)

MAKNA 'REPRESENTATIONAL STATE TRANSFER':

Saat klien meminta data mahasiswa, server mengirimkan 'representasi' (dalam wujud JSON/XML) dari 'state' (kondisi terkini) baris data di database server.

Mekanisme Pertukaran State pada REST:

```
[Database Server (Minggu 6)]
  Tabel courses: id=1, code='12S3101', quota=45
          |
          ▼ (State direpresentasikan ke JSON)
[REST Web Service]
  HTTP/1.1 200 OK
  Content-Type: application/json
  {
    "id": 1,
    "code": "12S3101",
    "quota": 45
  } ──► Dikirim ke Client (Mobile / Frontend)
```

SEGITIGA ARSITEKTUR REST: NOUNS, VERBS, REPRESENTATIONS

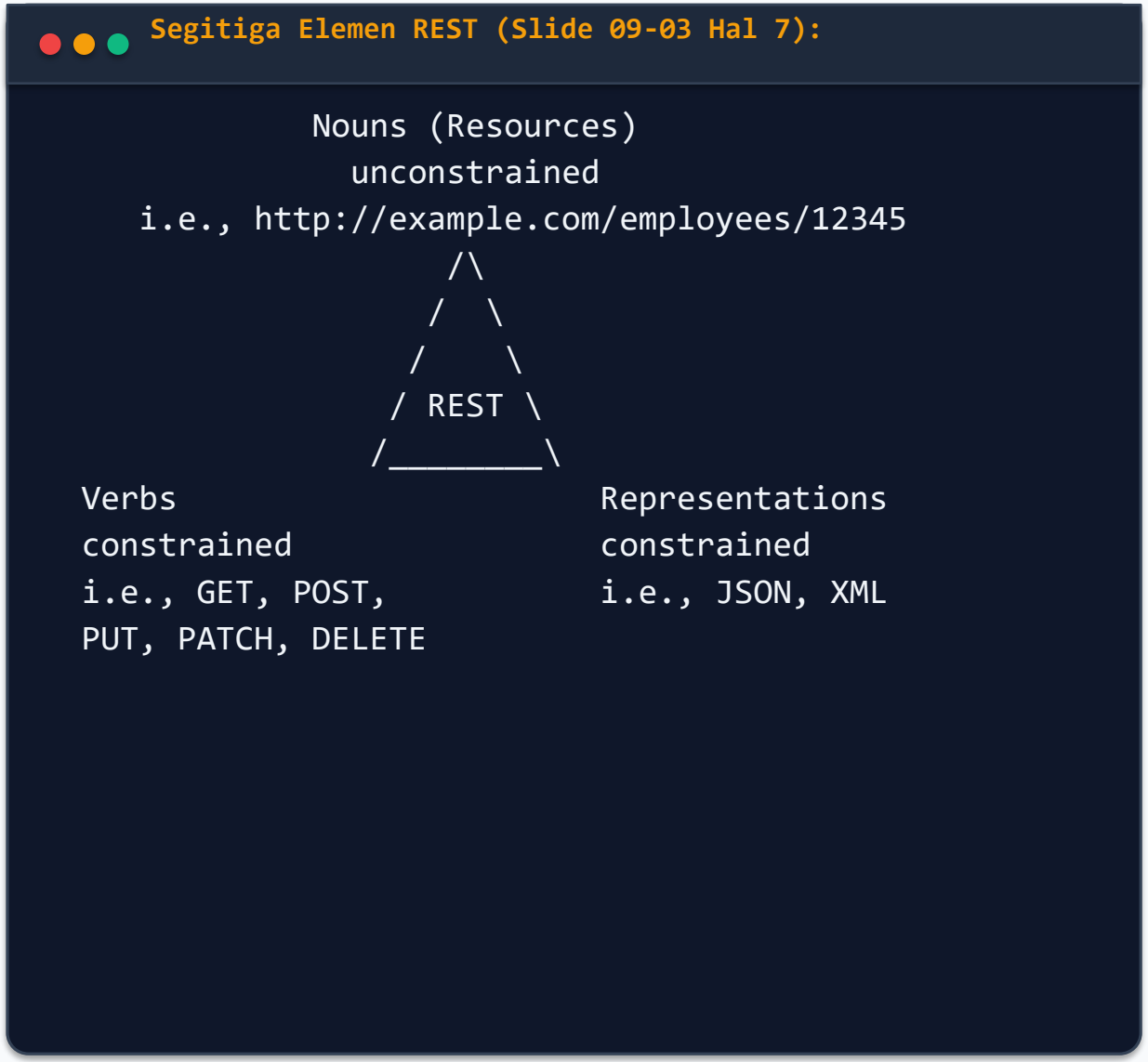
POIN PEMBAHASAN UTAMA:

- 1. Nouns (Resources): Objek data yang diidentifikasi lewat URI
- URI bersifat unconstrained (bebas dirancang sesuai entitas bisnis)
- Contoh Resource URI: http://example.com/employees/12345
- 2. Verbs (Kata Kerja): Terbatas (constrained) pada metode HTTP
- 3. Representations: Terbatas pada format media standar (JSON/XML)

PRINSIP PENAMAAN RESOURCE (NOUNS):

Dalam REST, URL menunjuk pada 'Kata Benda' (Resource seperti /employees atau /courses), bukan nama fungsi kerja seperti /getEmployeeById.

Segitiga Elemen REST (Slide 09-03 Hal 7):



SEMANTIK REST: PEMANFAATAN HTTP VERBS (METHODS)

POIN PEMBAHASAN UTAMA:

- Dibanding pendekatan tradisional (SOAP), REST menonjolkan semantik
- Semantik berarti request & response mampu 'self-explain'
- Dicapai dengan memaksimalkan HTTP Verbs yang ada di protokol HTTP
- GET (baca), POST (buat baru), PUT (ganti utuh), PATCH (ubah parsial)
- DELETE (hapus resource), serta OPTIONS (cek kapabilitas metode)

KORELASI DENGAN MINGGU 1, 2, & 5:

Di Minggu 2 form HTML hanya mengenal GET & POST (sehingga Laravel memakai @method('PUT') di Minggu 5). Pada REST API, klien dapat mengirim seluruh HTTP Verbs secara asli!

Perbandingan Semantik SOAP vs REST HTTP Verbs:

```
// Pendekatan Tradisional (SOAP):
POST /StockService  (Ambil data pakai POST!)
POST /StockService  (Hapus data juga pakai POST!)

// Pendekatan Semantik REST (Self-Explaining):
GET    /api/employees/12345  → Ambil data pegawai
POST   /api/employees        → Tambah pegawai baru
PUT    /api/employees/12345  → Ganti seluruh data
PATCH  /api/employees/12345  → Update parsial
DELETE /api/employees/12345  → Hapus data pegawai
```

SEMANTIK REST: KODE STATUS RESPONS HTTP

POIN PEMBAHASAN UTAMA:

- REST memakai HTTP Response Code untuk menyatakan hasil (outcome)
- Kategori 2xx (Success): 200 OK (berhasil), 201 Created (terbuat)
- Kategori 4xx (Client Error): 400 Bad Request, 401 Unauthorized
- 403 Forbidden (akses ditolak), 404 Not Found (tidak ditemukan)
- Kategori 5xx (Server Error): 500 Internal Server Error (gagal DB)

JANGAN KIRIM 200 OK SAAT TERJADI ERROR!

Anti-pattern yang sering terjadi adalah mengirim status 200 OK padahal isi JSON-nya {'error': 'Not Found'}. Gunakan kode status HTTP yang semantik (404)!

Daftar HTTP Response Code Utama pada REST:

```
200 OK              → Request sukses (GET/PATCH)
201 Created         → Resource baru berhasil dibuat
204 No Content      → Sukses DELETE tanpa body
400 Bad Request     → Sintaks JSON/parameter salah
401 Unauthorized    → Belum menyertakan token valid
403 Forbidden       → Token valid, tapi tak berhak
404 Not Found       → URI resource tidak ditemukan
422 Unprocessable   → Gagal validasi input bisnis
500 Internal Error  → Exception di sisi server
```

REST BERSIFAT STATELESS (KORELASI MINGGU 4)

POIN PEMBAHASAN UTAMA:

- Interaksi antara klien dan server pada REST bersifat Stateless
- Mengikuti sifat dasar protokol HTTP yang stateless
- Tidak ada sesi (No session at all) yang disimpan di memori server
- Klien adalah satu-satunya pihak yang menyimpan state sesinya
- Untuk data sensitif, klien wajib melampirkan Token tiap request

MENGAPA STATELESSNESS SANGAT MENGUNTUNGKAN?

Karena server tidak perlu menyimpan tab sesi untuk tiap user, beban RAM server sangat ringan dan aplikasi mudah diperbanyak (Horizontal Scalability / Load Balancer)!

Stateful Session (Minggu 4) vs Stateless REST:

```
// 1. Stateful Web (Minggu 4):
Server simpan file/memori Session tiap User.
Jika ada 100.000 user -> RAM Server terbebani!

// 2. Stateless REST (Minggu 9):
Server TIDAK menyimpan session user sama sekali.
Setiap Request berdiri mandiri membawa Token:
  GET /api/user HTTP/1.1
  Authorization: Bearer ghp_3c468c3851185...
  |
  ├─► Bisa dilayani Server #1, #2, atau #3!
```

DI MANA KONTRAK PADA LAYANAN BERBASIS REST?

POIN PEMBAHASAN UTAMA:

- Kontrak tetap esensial dan setiap service wajib mematuhi kontrak
- Pada layanan tradisional (SOAP), kontrak ditulis di dokumen WSDL
- Bagaimana dengan REST? Spesifikasi ditulis dalam Dokumentasi API
- Bukan berupa file spesifikasi kaku penurun kode secara otomatis
- Kelebihan: menyederhanakan pembuatan; Risiko: rawan inkonsistensi

SOLUSI MODERN: OPENAPI / SWAGGER SPECIFICATION:

Untuk mencegah inkonsistensi antara dokumentasi API dan kode program REST, industri modern kini banyak menggunakan standar OpenAPI (Swagger) YAML/JSON.

Perbandingan Kontrak SOAP vs Kontrak REST:

ASPEK KONTRAK	SOAP vs REST
Bentuk di SOAP	File XML WSDL Formal
Sifat WSDL SOAP	Wajib (Generate Stub)
Bentuk di REST	Dokumentasi API (Docs)
Keunggulan REST	Sangat ringkas & luwes
Tantangan REST	Rawan inkonsisten jika dokumen lupa di-update

PERBANDINGAN UTUH: SOAP VS REST-BASED SERVICES

POIN PEMBAHASAN UTAMA:

- Hakikat: SOAP adalah protokol W3C; REST adalah gaya arsitektur
- Format Pesan: SOAP wajib XML Envelope; REST fleksibel (JSON)
- Kontrak: SOAP wajib memakai WSDL; REST memakai Dokumentasi API
- Semantik: SOAP umumnya tunneling POST; REST memakai HTTP Verbs
- Performa & Skalabilitas: REST jauh lebih ringan dan stateless

PEDOMAN PEMILIHAN ARSITEKTUR:

Gunakan REST (JSON) untuk aplikasi web modern, mobile app, dan microservices; gunakan SOAP hanya ketika berintegrasi dengan sistem enterprise legacy yang mewajibkannya.

Matriks Ringkasan SOAP vs REST:

KRITERIA	SOAP SERVICE	REST SERVICE
Hakikat	Protokol (W3C)	Arsitektur Web
Format Data	Hanya XML	JSON, XML, dll
Kontrak API	WSDL (Wajib)	API Docs
HTTP Verbs	Hanya POST	GET,POST,PATCH
Sifat State	Bisa Stateful	Stateless
Ukuran Paket	Besar (Verbose)	Kecil & Cepat
Target Utama	Sistem Legacy	Web & Mobile

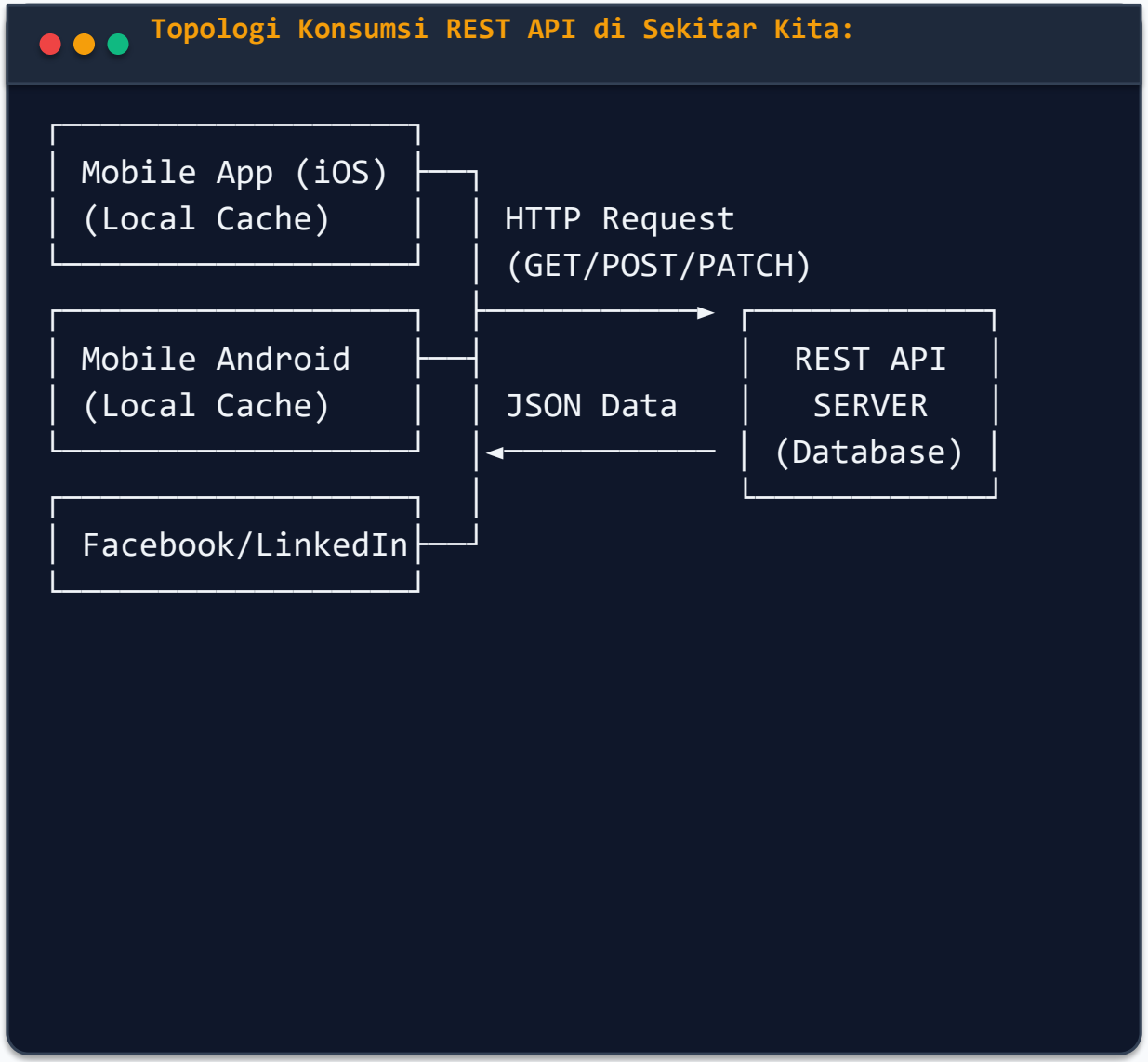
REST SERVICES DI SEKITAR KITA: MOBILE & MEDIA SOSIAL

POIN PEMBAHASAN UTAMA:

- Aplikasi mobile tidak menyimpan seluruh isi datanya di HP lokal
- Mayoritas data disimpan di server; HP hanya simpan cache terbaru
- Komunikasi antara aplikasi mobile dan server menggunakan REST
- Situs media sosial (Facebook, LinkedIn) memuat data dinamis
- Pengambilan data di latar belakang berjalan mulus via REST API

ARSITEKTUR MULTI-PLATFORM SATU BACKEND:

Dengan menyediakan satu REST API di server, data yang sama dapat dikonsumsi serentak oleh Website, aplikasi Android, dan aplikasi iOS.



MEMBEDAH DOKUMENTASI KONTRAK GITHUB REST API

POIN PEMBAHASAN UTAMA:

- GitHub menyediakan REST API publik untuk akses data repo & user
- Dokumentasi kontrak: docs.github.com/en/.../rest/reference/users
- Operasi 'Get a user' mengambil informasi publik akun seseorang
- Dari kontrak terlihat Verb = GET dan URL = /users/{username}
- Spesifikasi Parameter: header 'accept' & path param 'username'

MEMBACA KONTRAK SEBELUM MENULIS KODE:

Perhatikan bagaimana dokumentasi GitHub memenuhi 3 syarat kontrak: cara konsumsi (GET /users/{username}), parameter input, dan contoh JSON output.

Ekstraksi Kontrak 'Get a user' (Slide 09-03):

```
[Dokumentasi Kontrak GitHub API: Get a user]
_____________________________________________

• HTTP Verb   : GET
• Base URL    : https://api.github.com
• Path URI    : /users/{username}

• Parameters :
  1. accept   (string | in: header)
     Value: application/vnd.github.v3+json
  2. username (string | in: path)
     Handle akun GitHub yang ingin dicari.

• Contoh cURL Resmi di Dokumentasi:
  curl -H "Accept: application/vnd.github.v3+json"
       https://api.github.com/users/USERNAME
```

KONSUMSI GITHUB API VIA CURL & WEB BROWSER

POIN PEMBAHASAN UTAMA:

- Cara 1 (Terminal CLI): Mengonsumsi endpoint memakai utilitas cURL
- Sertakan flag -H untuk mengirim header Accept sesuai kontrak
- Target URL: https://api.github.com/users/sigurita
- Cara 2 (Web Browser): Buka URL tersebut di address bar browser
- Mengapa bisa lewat browser? Karena endpoint ini memakai verb GET!

KETERBATASAN WEB BROWSER BIASA:

Address bar browser hanya dapat memicu metode HTTP GET tanpa kustomisasi header otorisasi. Untuk menguji POST, PATCH, atau DELETE, kita butuh API Testing Tool!

Konsumsi via cURL & Output JSON (Slide 09-03):

```
C:\Users\CHP> curl \
  -H "Accept: application/vnd.github.v3+json" \
  https://api.github.com/users/sigurita

{
  "login": "sigurita",
  "id": 35382893,
  "url": "https://api.github.com/users/sigurita",
  "html_url": "https://github.com/sigurita",
  "type": "User",
  "name": "sigurita",
  "blog": "https://sigurita.com",
  "bio": null,
  "public_repos": 2
}
```

PENGUJIAN DENGAN API TESTING TOOL (SOAPUI & POSTMAN)

POIN PEMBAHASAN UTAMA:

- Gunakan API testing tool khusus seperti SoapUI atau Postman
- Mendukung semua HTTP Method (GET, POST, PUT, PATCH, DELETE)
- Memudahkan pengaturan Header, Parameter, Body JSON, & Auth
- Menampilkan Raw HTTP Response lengkap (Status Line + Headers)
- Terlihat jelas: HTTP/1.1 200 OK & content-type: application/json

PERSIAPAN PENGUJIAN API (MINGGU 11–12):

Tools seperti Postman dan SoapUI akan menjadi senjata utama kita saat merancang skenario pengujian otomatis (assertions) pada materi API Testing mendatang.

Inspeksi Raw Response di SoapUI (Slide 09-03):

```
Method   : GET
Endpoint : https://api.github.com/users/sigurita
Header   : Accept = application/vnd.github.v3+json
```

```
[Response (Raw) pada Endpoint Explorer SoapUI]:
HTTP/1.1 200 OK
date: Thu, 12 Nov 2020 02:40:50 GMT
content-type: application/json; charset=utf-8
server: GitHub.com
status: 200 OK
cache-control: public, max-age=60, s-maxage=60
x-github-media-type: github.v3; format=json
```

AUTHORIZED ACCESS & PERSONAL ACCESS TOKENS

POIN PEMBAHASAN UTAMA:

- Ada situasi di mana API mewajibkan konsumen terotorisasi dulu
- Contoh: Mengubah biodata profil atau menghapus repositori user
- Karena REST bersifat stateless, otorisasi diwujudkan lewat Token
- Token wajib dilampirkan (attached) pada setiap pengiriman request
- Di GitHub: Settings > Developer Settings > Personal access tokens

ATURAN KEAMANAN MUTLAK: DO NOT SHARE THE TOKEN!

Token akses memiliki hak setara password akun Anda (sesuai scope yang dicentang seperti 'user'). Jangan pernah mem-push token ke repositori publik Git!

Konfigurasi Personal Access Token (Slide 09-03):

```
[GitHub: Settings > Developer Settings > Tokens]
Pilih Scopes (Hak Akses Token):
  [✓] user           (Update all user data)
      [✓] read:user  (Read all user profile data)
      [✓] user:email (Access user email addresses)

Token yang Dihasilkan:
  ✓ 3c468c385118511063329938 74ce0d97c7b*****

PERINGATAN KERAS:
  "Do NOT share the token!"
  Header: Authorization: Bearer <token>
```

MENGUBAH DATA PROFIL DENGAN PATCH /USER & TOKEN

POIN PEMBAHASAN UTAMA:

- Kontrak GitHub: 'Update the authenticated user' memakai PATCH
- Endpoint target: PATCH https://api.github.com/user
- Parameter body yang dikirim: field 'bio' bertipe string (JSON)
- Di SoapUI/Postman, isi Access Token pada tab Auth (OAuth 2)
- Hasil: Status 200 OK, field 'bio' berubah dari null jadi 'Aloha'!

MENGAPA MENGGUNAKAN VERB PATCH (BUKAN PUT)?

Karena kita hanya ingin memperbarui satu atribut saja ('bio') tanpa menimpa atribut profil lainnya (seperti 'name', 'blog', atau 'company').

Eksekusi PATCH /user dengan Token (Slide 09-03):

```
// 1. Request via cURL / SoapUI / Postman:
curl -X PATCH https://api.github.com/user \
  -H "Accept: application/vnd.github.v3+json" \
  -H "Authorization: Bearer 3c468c3851185..." \
  -d '{"bio": "Aloha"}'

// 2. Response JSON dari GitHub Server (200 OK):
{
  "login": "sigurita",
  "name": "sigurita",
  "blog": "https://sigurita.com",
  "bio": "Aloha",  // <-- Berhasil ter-update!
  "plan": { "name": "free" }
}
```

HUBUNGAN DENGAN LARAVEL 11: PROVIDER & CONSUMER REST

POIN PEMBAHASAN UTAMA:

- Bagaimana menghubungkan materi Minggu 5–6 dengan Minggu 9?
- Sebagai Provider: Model Eloquent diekspos lewat routes/api.php
- Controller mereturn response()->json($data, 200) sesuai kontrak
- Sebagai Consumer: Laravel memanggil API luar via jalur Http
- Method Http::withToken() mengirim Bearer Token ke GitHub API

SATU MODEL ELOQUENT, DUA KANAL LAYANAN:

Model Course dan transaksi database yang kita bangun di Minggu 6 dapat melayani halaman web Blade (Minggu 5) sekaligus melayani aplikasi Mobile via REST API (Minggu 9)!

Laravel 11 sebagai REST Provider & Consumer:

```
// 1. LARAVEL SEBAGAI REST PROVIDER (api.php):
Route::get('/courses', function () {
    return response()->json(Course::all(), 200);
});

// 2. LARAVEL SEBAGAI REST CONSUMER (Http Client):
use Illuminate\Support\Facades\Http;

$res = Http::withToken(env('GITHUB_TOKEN'))
    ->accept('application/vnd.github.v3+json')
    ->patch('https://api.github.com/user', [
        'bio' => 'Aloha dari Laravel 11!',
    ]);
```

PETA KONSEP MINGGU 9 & PERSIAPAN MINGGU KE-10

POIN PEMBAHASAN UTAMA:

- Web Service menjembatani pertukaran data & kapabilitas aplikasi
- Kontrak API adalah kesepakatan mutlak antara Provider & Consumer
- SOAP mengandalkan XML Envelope & kontrak WSDL (sistem legacy)
- REST memaksimalkan semantik HTTP (Verbs & Status) secara stateless
- Minggu 10: Kita akan membedah perancangan semantik REST lebih dalam

AGENDA PERTEMUAN BERIKUTNYA (MINGGU 10):

Di Minggu 10 kita akan merancang spesifikasi endpoint REST API secara mendalam: struktur URI, matriks HTTP Verbs, Status Codes, JSON Payload, dan HATEOAS.

Alur Kelanjutan Materi Minggu 9 ke Minggu 12:

```
[Minggu 09] Konsep Web Services, SOAP, & REST
      |
      ▼
[Minggu 10] Perancangan Kontrak & Semantik REST
      |       (URI Design, Verbs, Status, HATEOAS)
      ▼
[Minggu 11] Implementasi REST Provider & Consumer
      |
      ▼
[Minggu 12] Pengujian REST API (Postman & Test)
```

TO-DOS & TUGAS EKSPLORASI MANDIRI MINGGU KE-9

POIN PEMBAHASAN UTAMA:

- 1. Eksplorasi dokumentasi resmi GitHub API (docs.github.com/rest)
- 2. Uji konsumsi endpoint publik GET /users/{user} via cURL/Browser
- 3. Gunakan Postman / SoapUI untuk inspeksi Raw Response Header
- 4. Buat Personal Access Token GitHub & uji update bio via PATCH
- 5. Identifikasi kandidat REST API untuk proyek kelompok Anda

PENGINGAT KEAMANAN PRAKTIKUM:

Saat mengumpulkan laporan tangkapan layar (screenshot) praktikum pengujian API, pastikan sebagian karakter Personal Access Token Anda disensor dan cabut (revoke) token setelah selesai.

Checklist To-Dos Mahasiswa Minggu 9:

```
[✓] Memahami perbedaan peran Consumer vs Provider
[✓] Memahami anatomi SOAP Envelope & kontrak WSDL
[✓] Menguji GET https://api.github.com/users/{anda}
[✓] Mengamati Status Code 200 OK & Header JSON
[✓] Menguji PATCH /user dengan Bearer Access Token
[✓] Menyusun daftar kebutuhan REST API Proyek
```

REFERENSI RESMI & SESI DISKUSI TANYA JAWAB

POIN PEMBAHASAN UTAMA:

- Srinivasan, M. (2012). Web Technology: Theory and Practice.
- Erl, T. (2016). Service-Oriented Architecture: Analysis & Design.
- Richardson, L. et al. (2013). RESTful Web APIs. O'Reilly Media.
- Fielding, R. T. (2000). Architectural Styles & Network Software.
- GitHub REST API Documentation: https://docs.github.com/en/rest

PESAN PENUTUP MINGGU 9:

'Tidak ada aplikasi yang hidup menyendiri. Kekuatan sistem informasi modern terletak pada kemampuannya berkolaborasi melalui kontrak layanan (API) yang jelas dan aman.'

Referensi Utama & Tautan Dokumentasi:

[1] W3C SOAP Version 1.2 Specification:
https://www.w3.org/TR/soap12/

[2] GitHub REST API Official Documentation:
https://docs.github.com/en/rest

[3] Postman Learning Center & SoapUI Docs:
https://learning.postman.com/

Terima Kasih! Silakan ajukan pertanyaan.